

第 10 章

信息隐藏与数字水印实验

10.1 信号处理基础

【实验目的】

了解音频和图像数据系数特点,掌握音频和图像文件的离散傅里叶、离散余弦和离散小波变换等基本操作。

【实验环境】

- (1) WindowsXP 或 Vista 操作系统;
- (2) Matlab7.1 版本软件;
- (3) BMP 格式图像文件;
- (4) Wav 格式音频文件。

【原理简介】

离散傅里叶、离散余弦和离散小波变换是图像、音频信号常用基础操作,时域信号转换到不同变换域以后,会导致不同程度的能量集中,信息隐藏利用这个原理在变换域选择适当位置系数进行修改,嵌入信息,并确保图像、音频信号经处理后感官质量无明显变化。

- (1) 一维离散傅里叶变换对定义

一维离散傅里叶变换:

$$X(k) = \sum_{n=0}^{N-1} x(n)e^{j2\pi kn/N}$$

一维离散傅里叶逆变换:

$$n(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k)e^{j2\pi kn/N}$$

- (2) 一维离散余弦变换对定义

一维离散余弦正变换:

$$C(0) = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} f(x)$$
$$C(u) = \sqrt{\frac{2}{N}} \sum_{x=0}^{N-1} f(x) \cos \frac{(2x+1)u\pi}{2N}, \quad u = 1, 2, \dots, N-1$$

信息隐藏与数字水印

一维离散余弦逆变换:

$$f(x) = \frac{1}{\sqrt{N}}C(0) + \sqrt{\frac{2}{n}} \sum_{u=1}^{N-1} C(u) \cos \frac{(2x+1)u\pi}{2N}, \quad u = 0, 1, \dots, N-1$$

(3) 一维连续小波变换对定义

一维连续小波变换:

$$\text{CWT}_x(\tau, a) = \frac{1}{\sqrt{|a|}} \int x(t) h^* \left(\frac{t-\tau}{a} \right) dt$$

其中, $h(t)$ 是小波母函数。

一维连续小波逆变换:

$$x(t) = \frac{1}{C_H} \iint \frac{1}{a^2} \text{CWT}_x(\tau, a) \frac{1}{\sqrt{|a|}} h \left(\frac{t-\tau}{a} \right) da db$$

(4) 二维离散傅里叶变换对定义

二维离散傅里叶变换:

$$F(u, v) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-j2\pi(ux/M + vy/N)}$$

其中, $u=0, 1, \dots, M-1; v=0, 1, \dots, N-1$ 。

二维离散傅里叶逆变换:

$$f(x, y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{j2\pi(ux/M + vy/N)}$$

其中, $x=0, 1, \dots, M-1; y=0, 1, \dots, N-1$ 。

(5) 二维离散余弦变换对定义

二维离散余弦正变换:

$$C(0, 0) = \frac{1}{N} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y)$$
$$C(u, v) = \frac{2}{N} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) \cos \frac{(2x+1)\pi u}{2N} \cos \frac{(2y+1)\pi v}{2N}$$

二维离散余弦逆变换:

$$f(x, y) = \frac{1}{N} C(0, 0) + \frac{2}{N} \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} C(u, v) \cos \frac{(2x+1)\pi u}{2N} \cos \frac{(2y+1)\pi v}{2N}$$

【实验步骤】

1. 用离散傅里叶变换分析合成音频和图像分析合成音频文件包括以下步骤:

- 读取音频文件数据;
- 一维离散傅里叶变换;
- 一维离散傅里叶逆变换;
- 观察结果。

第一步: 读取音频文件数据。

新建一个 m 文件, 另存为 example11.m, 输入以下命令:

clc;

```
clear;
len = 40000;
[fn, pn] = uigetfile('*.wav', '请选择音频文件');
[x, fs] = wavread(strcat(pn, fn), len);
```

uigetfile 是文件对话框函数, 提供图形界面供用户选择所需文件, 返回目标的目录名和文件名。

函数原型: `y = wavread(FILE)`。

功能: 读取微软音频格式(Wav)文件内容。

输入参数: file 表示音频文件名, 字符串。

返回参数: y 表示音频样点, 浮点型。

第二步: 一维离散傅里叶变换。

新建一个 m 文件, 另存为 example12.m, 输入以下命令:

```
xf = fft(x);
f1 = [0:len-1] * fs / len;
xff = fftshift(xf);
h1 = floor(len / 2);
f2 = [-h1:h1] * fs / len;
```

fft 函数对输入参数进行一维离散傅里叶变换并返回其系数, 对应频率从 0 到 f_s (采样频率), 使用 fftshift 将零频对应系数移至中央。上述代码还计算了离散样点对应的频率值, 以便更好地观察频谱。

第三步: 一维离散傅里叶逆变换。

新建一个 m 文件, 另存为 example13.m, 输入以下命令:

```
xsync = ifft(xf);
```

ifft 函数对输入参数进行一维离散傅里叶逆变换并返回其系数。

第四步: 观察结果。

新建一个 m 文件, 另存为 example14.m, 输入以下命令:

```
figure;
subplot(2, 2, 1); plot(x); title('original audio');
subplot(2, 2, 2); plot(xsync); title('synthesize audio');
subplot(2, 2, 3); plot(f1, abs(xf)); title('fft coef. of audio');
subplot(2, 2, 4); plot(f2(1:len), abs(xff)); title('fftshift coef. of auio');
figure(n) 表示创建第 n 个图形窗。
```

subplot 是子绘图函数, 第一、二个参数指明子图像布局方式, 例如, 若参数为 2, 3 则表示画面共分为 2 行, 每行有 3 个子图像。第三个参数表明子图像序号, 排序顺序为从左至右, 从上至下。

plot 是绘图函数, 默认使用方式为 plot(y), 参数 y 是要绘制的数据; 如果需要指明图像横轴显示序列, 则命令行为 plot(x, y), 默认方式等同于 plot([0:len-1], y), len 为序列 y 的长度。

用离散傅里叶变换分析合成音频文件如图 10-1 所示。

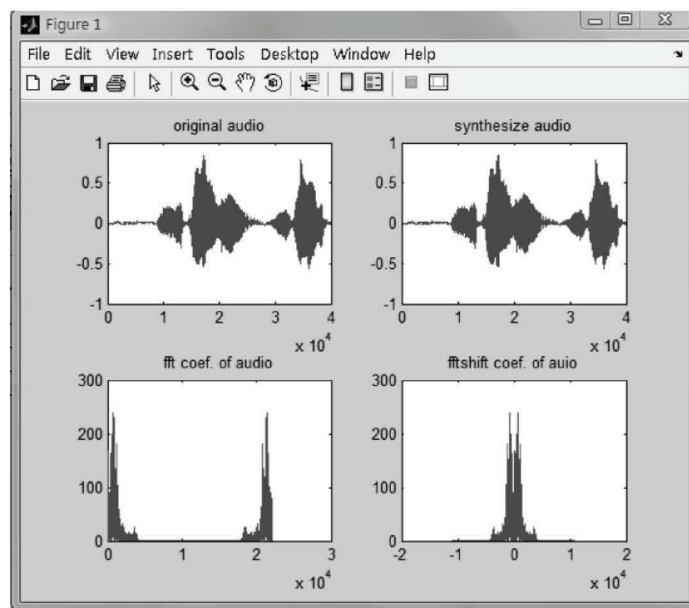


图 10-1 用离散傅里叶变换分析合成音频文件

分析合成图像文件包括以下步骤：

- 读取图像文件数据；
- 二维离散傅里叶变换；
- 二维离散傅里叶逆变换；
- 观察结果。

第一步：读取图像文件数据。

新建一个 m 文件，另存为 example21.m，输入以下命令：

```
[fn, pn] = uigetfile('*.bmp', '请选择图像文件');
[x, map] = imread(strcat(pn, fn), 'bmp');
I = rgb2gray(x);
```

函数原型： $A = \text{imread}(\text{filename}, \text{fmt})$ 。

功能：读取 fmt 指定格式的图像文件内容。

输入参数：filename 表示图像文件名，字符串。

Fmt 表示图像文件格式名，字符串、函数支持的图像格式包括：JPEG, TIFF, GIF, BMP 等，当参数中不包括文件格式名时，函数尝试推断出文件格式。

返回参数：A 表示图像数据内容，整型。

rgb2gray 将 RGB 图像转换为灰度图。

第二步：二维离散傅里叶变换。

新建一个 m 文件，另存为 example22.m，输入以下命令：

```
xf = fft2(I);
xff = fftshift(xf);
```

fft2 函数对输入参数进行二维离散傅里叶变换并返回其系数，使用 fftshift 将零频对应

系数移至中央。

第三步:二维离散傅里叶逆变换。

新建一个 m 文件,另存为 example23.m,输入以下命令:

```
xsync = ifft2(xf);
```

ifft2 函数对输入参数进行二维离散傅里叶逆变换并返回其系数。

第四步:观察结果。

新建一个 m 文件,另存为 example24.m,输入以下命令:

```
figure;
```

```
subplot(2, 2, 1);imshow(x);title('original image');
```

```
subplot(2, 2, 2);imshow(uint8(abs(xsync)));title('synthesize image');
```

```
subplot(2, 2, 3);mesh(abs(xf));title('fft coef. of image');
```

```
subplot(2, 2, 4);mesh(abs(xff));title('fftshift coef. of image');
```

imshow 是二维数据绘图函数, mesh 通过三维平面显示数据。

用离散傅里叶变换分析合成图像文件如图 10-2 所示。

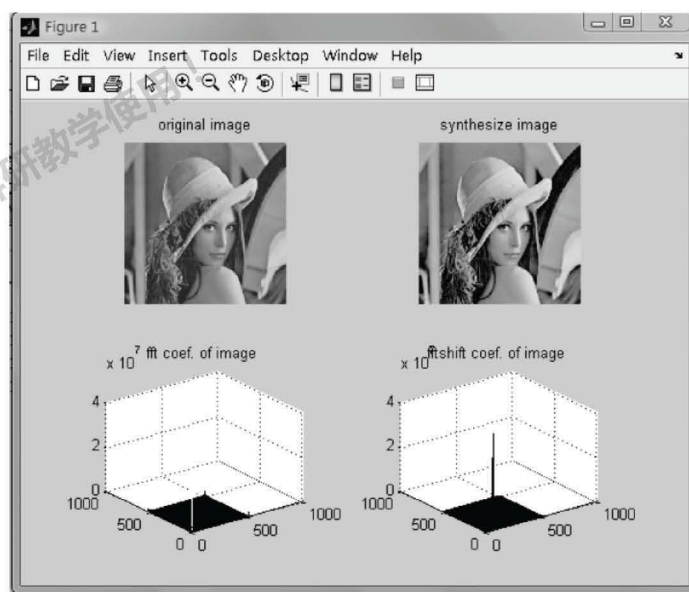


图 10-2 用离散傅里叶变换分析合成图像文件

2. 用离散余弦变换分析合成音频和图像

分析合成音频文件包括以下步骤:

- 读取音频文件数据;
- 一维离散余弦变换;
- 一维离散余弦逆变换;
- 观察结果。

第一步:一维离散余弦变换。

新建一个 m 文件,另存为 example31.m,输入以下命令:

信息隐藏与数字水印

```
xf = dct(x);
```

dct 函数对输入参数进行一维离散余弦变换并返回其系数,对应频率从 0 到 f_s (采样频率)。

第二步:一维离散余弦逆变换。

新建一个 m 文件,另存为 example32.m,输入以下命令:

```
xsync = idct(xf);
```

```
[row,col]=size(x);
```

```
xff=zeros(row,col);
```

```
xff(1:row,1:col)=xf(1:row,1:col);
```

```
y=idct(xff);
```

idct 函数对输入参数进行一维离散余弦逆变换并返回其系数。离散余弦变换常用于图像压缩,可以尝试只使用部分系数重构语言,通过观察可发现,原始音频和合成后音频两者差别不大。

第三步:观察结果。

新建一个 m 文件,另存为 example33.m,输入以下命令:

```
figure;
```

```
subplot(2,2,1);plot(x);title('original audio');
```

```
subplot(2,2,2);plot(xsync);title('synthesize audio');
```

```
subplot(2,2,3);plot(f1,abs(xf));title('fft coef. of audio');
```

```
subplot(2,2,4);plot(f2(1:len),abs(xff));title('fftshift coef. of auio');
```

用离散余弦变换分析合成音频文件如图 10-3 所示。

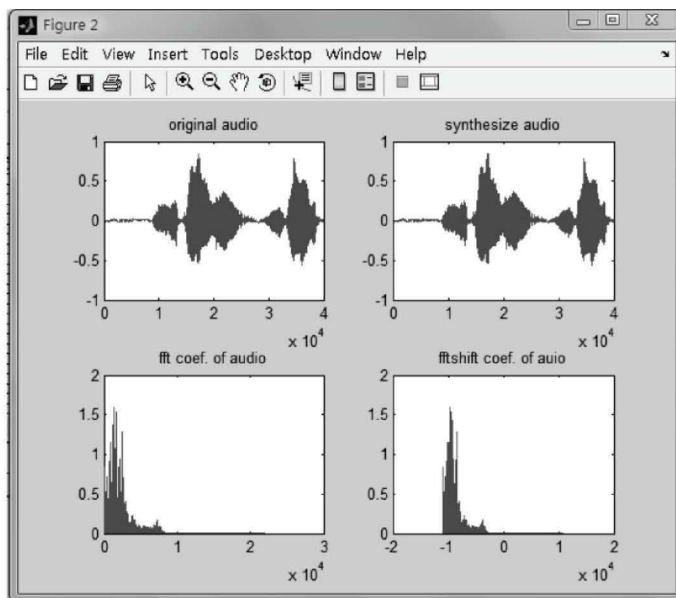


图 10-3 用离散余弦变换分析合成音频文件

分析合成图像文件包括以下步骤：

- 读取图像文件数据；
- 二维离散余弦变换；
- 二维离散余弦逆变换；
- 观察结果。

第一步：二维离散余弦变换。

新建一个 m 文件，另存为 example41.m，输入以下命令：

```
xf = dct2(I);
```

dct2 函数对输入参数进行二维离散余弦变换并返回其系数。

第二步：二维离散余弦逆变换。

新建一个 m 文件，另存为 example42.m，输入以下命令：

```
xsync = uint8(idct2(xf));
[ row, col ] = size(I);
lenr = round(row * 4 / 5);
lenc = round(col * 4 / 5);
xff = zeros(row, col);
xff(1:lenr, 1:lenc) = xf(1:lenr, 1:lenc);
y = uint8(idct2(xff));
```

idct2 函数对输入参数进行二维离散余弦逆变换并返回其系数。可以尝试使用部分系数重构图像，本例中使用了系数矩阵中 4/5 的数据，其他部分置零。为了保证图像能正确显示，使用 uint8 对重构图像原始数据进行了数据类型转换，确保其取值范围在 0~255 之间。

第三步：观察结果。

请输入命令显示四个子图，分别是原始图像、使用全部系数恢复的图像、使用部分系数恢复的图像和用三维立体图方式显示系数。

新建一个 m 文件，另存为 example43.m，输入以下命令：

```
figure;
subplot(2, 2, 1); imshow(x); title('original image');
subplot(2, 2, 2); imshow(uint8(abs(xsync))); title('synthesize image');
subplot(2, 2, 3); imshow(uint8(abs(y))); title('part synthesize image');
subplot(2, 2, 4); mesh(abs(xff)); title('fftshift coef. of image');
```

用离散余弦变换分析合成图像文件如图 10-4 所示。

3. 用离散小波变换分析合成音频和图像

分析合成音频文件包括以下步骤：

- 读取音频文件数据；
- 一维离散小波变换；
- 一维离散小波逆变换；
- 观察结果。

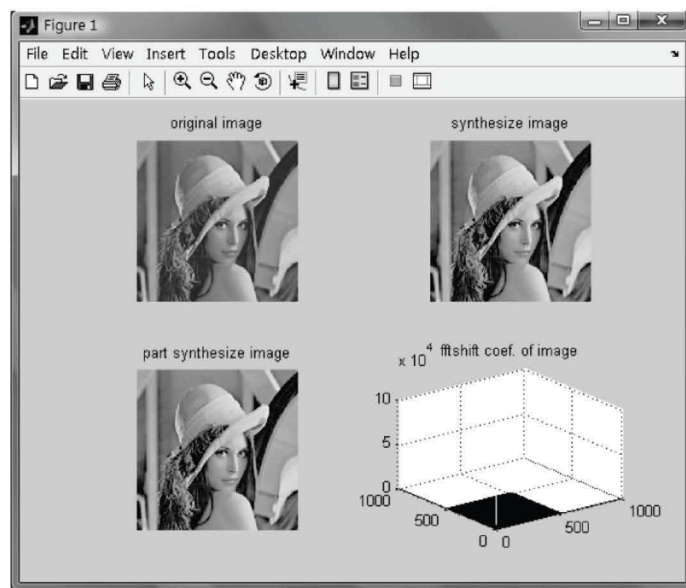


图 10-4 用离散余弦变换分析合成图像文件

第一步：一维离散小波变换。

新建一个 m 文件，另存为 example51.m，输入以下命令：

```
[C, L] = wavedec(x, 2, 'db4');
```

wavedec 函数对输入参数进行一维离散小波变换并返回其系数 C 和各级系数长度 L。

第二个参数指明小波变换的级数，第三个参数指明小波变换使用的小波基名称。

第二步：一维离散小波逆变换。

新建一个 m 文件，另存为 example52.m，输入以下命令：

```
xsync = waverec(C, L, 'db4');
```

```
cA2 = appcoef(C, L, 'db4', 2);
```

```
cD2 = detcoef(C, L, 2);
```

```
cD1 = detcoef(C, L, 1);
```

waverec 函数对输入参数进行一维离散小波逆变换并返回其系数。appcoef 返回小波系数近似分量，第一个参数 C、第二个参数 L 是 wavedec 的返回参数，为各级小波系数和其长度，第三个参数指明小波基名称，第四个参数指明级数。detcoef 返回小波系数细节分量，第一个参数 C、第二个参数 L 是 wavedec 的返回参数，为各级小波系数和其长度，第三个参数指明级数。

第三步：观察结果。

新建一个 m 文件，另存为 example53.m，输入以下命令：

```
figure;
```

```
subplot(2, 3, 1);plot(x);title('original audio');
```

```
subplot(2, 3, 2);plot(xsync);title('synthesize audio');
```

```
subplot(2, 3, 4);plot(cA2);title('app coef. of audio');
```

```
subplot(2, 3, 5);plot(cD2);title('det coef. of auio');
```

```
subplot(2, 3, 6);plot(cD1);title('det coef. of auio');
```

用离散小波变换分析合成音频文件如图 10-5 所示。

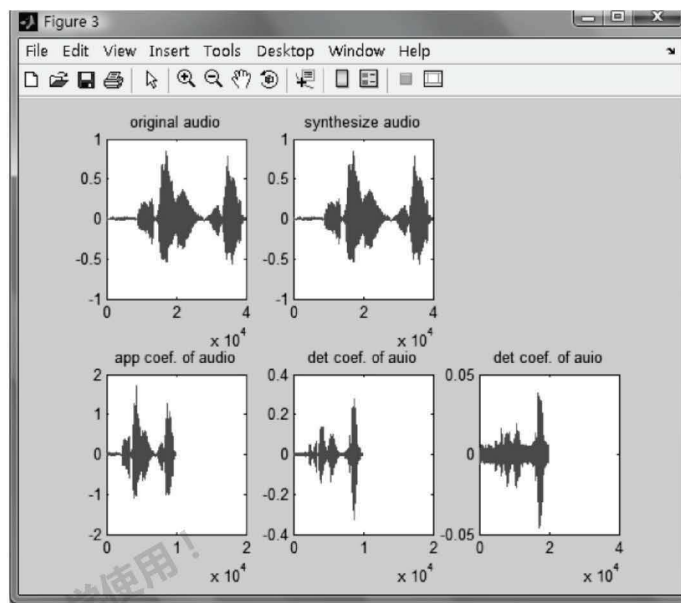


图 10-5 用离散小波变换分析合成音频文件

分析合成图像文件包括以下步骤：

- 读取图像文件数据；
- 二维离散小波变换；
- 二维离散小波逆变换；
- 观察结果。

第一步：二维离散小波变换。

新建一个 m 文件，另存为 example61.m，输入以下命令：

```
sx = size(I);
```

```
[cA1, cH1, cV1, cD1] = dwt2(I, 'bior3.7');
```

dwt2 函数对输入参数进行二维一级离散小波变换并返回近似分量，水平细节分量，垂直细节分量和对角线细节分量。如果要对图像进行多级小波分解，使用 wavedec2 函数。

第二步：二维离散小波逆变换。

新建一个 m 文件，另存为 example62.m，输入以下命令：

```
xsyrc = uint8(idwt2(cA1, cH1, cV1, cD1, 'bior3.7', sx));
```

```
A1 = uint8(idwt2(cA1, [], [], [], 'bior3.7', sx));
```

```
H1 = uint8(idwt2([], cH1, [], [], 'bior3.7', sx));
```

```
V1 = uint8(idwt2([], [], cV1, [], 'bior3.7', sx));
```

```
D1 = uint8(idwt2([], [], [], cD1, 'bior3.7', sx));
```


信息隐藏与数字水印

idwt2 函数对输入参数进行二维离散小波逆变换并返回其系数。可以尝试仅使用近似分量、水平细节分量、垂直细节分量或对角线细节分量重构图像。

第三步:观察结果。

输入命令显示六个子图,分别是原始图像、使用全部系数恢复的图像、小波系数近似分量、水平细节分量、垂直细节分量和对角线细节分量。

新建一个 m 文件,另存为 example63.m,输入以下命令:

```
figure;  
subplot(2, 3, 1);imshow(x);title('original image');  
subplot(2, 3, 2);imshow(uint8(abs(xsync)));title('synthesize image');  
subplot(2, 3, 3);mesh(A1);title('app coef. of image');  
subplot(2, 3, 4);mesh(H1);title('hor coef. of image');  
subplot(2, 3, 5);mesh(V1);title('ver coef. of image');  
subplot(2, 3, 6);mesh(D1);title('dia coef. of image');
```

用离散小波变换分析合成图像文件如图 10-6 所示。

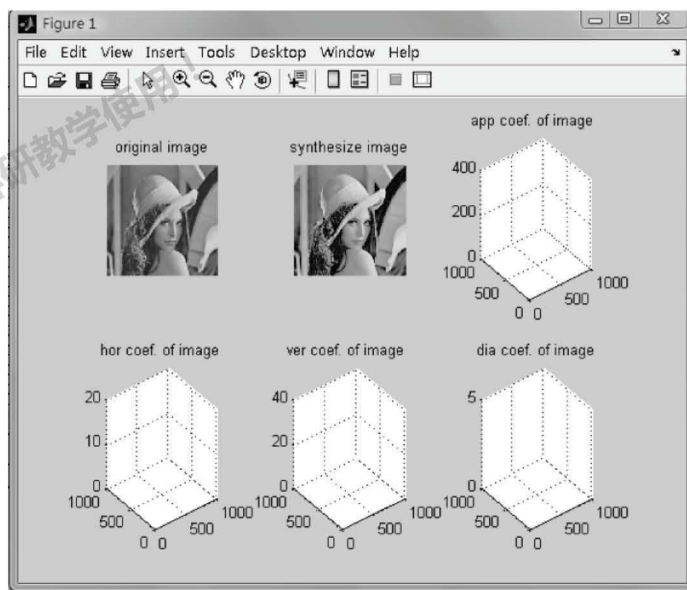


图 10-6 用离散小波变换分析合成图像文件

10.2 BMP 图像信息隐藏

【实验目的】

了解 BMP 图像文件格式,了解利用 BMP 图像文件隐藏信息的原理,设计并实现一种基于 24 位真彩色 BMP 图像的文件信息隐藏方法。

【实验环境】

- (1) Windows XP 或 Vista 操作系统;
- (2) Matlab7.1 版本软件;
- (3) BMP 格式图片文件;
- (4) Ultraedit 编辑工具。

【原理简介】

针对文件结构的信息隐藏方法需详细掌握文件的格式,利用文件结构块之间的关系或根据块数据和块大小之间的关系来隐藏信息。

BMP(Bitmap-File)图形文件是 Windows 采用的常见图形文件格式,要利用 BMP 位图进行信息隐藏首先需要详细了解 BMP 文件的格式,BMP 图像文件结构比较单一而且固定,BMP 图像由文件头、信息头、调色板区和数据区四个部分组成,而 24 位真彩色图像中没有调色板信息。24 位真彩色 BMP 位图文件包括 3 部分。第一部分是 BMP 文件头。前 2 个字节是“BM”,是用于识别 BMP 文件的标志;第 3~6 字节存放的是位图文件的大小,以字节为单位;第 7~10 字节是保留的,必须为 0;第 11~14 字节给出位图阵列相对于文件头的偏移,在 24 位真彩色图像中,这个值固定为 54;第 19~22 字节表示的是图像文件的宽度,以像素为单位;第 23~26 字节表示的是图像文件的高度,以像素为单位。第二部分是位图信息头。从第 29 个字节开始,第 29、30 字节描述的是像素的位数,24 位真彩色位图,该位的值为 0x18。第三部分是数据区。从第 55 个字节开始,每 3 个字节表示一个像素,这 3 个字节依次表示该像素的红、绿、蓝亮度分量值。

在不影响图像正常显示的情况下,可使用以下四种方法在 24 位真彩色 BMP 图像中隐藏信息。

- 在图像文件尾部添加任意长度的数据,秘密信息存放在文件尾部可以减少修改文件头的的数据量,仅需修改文件头中文件长度的值即可。
- 在调色板或者位图信息头和实际的图像数据之间隐藏数据,如果将秘密数据放在文件头与图像数据之间,则至少需要修改文件头中文件长度、数据起始偏移地址这两个域的值。
- 修改文件头和信息头中的保留字段隐藏信息。
- 在图像像素区利用图像宽度字节必须是 4 的倍数的特点,在补足位处隐藏数据。

【实验步骤】

1. 在实际的图像数据后隐藏信息

待隐藏的秘密信息文件名称为 hidden.txt, Baboon.bmp 为载体图像,将载体和秘密信息放置在同一个目录下,在 Windows 的 MS-DOS 方式下执行命令 Copy baboon.bmp /b + hidden.txt /a baboon1.bmp,其中参数/b 指定以二进制格式复制、合并文件,参数/a 以 ASCII 格式复制、合并文件。执行该命令后,生成一个新的 baboon1.bmp 文件,使用图像浏览工具浏览该文件发现与原始载体图像几乎完全相同,信息隐藏在 baboon.bmp 文件的尾部。从 BMP 图像的结构中可知,图像的 3~6 四个字节存放整个 BMP 图像的长度。使用

信息隐藏与数字水印

该方法隐藏信息时,未修改图像文件的文件长度字节,通过比较文件的实际长度和文件中保存的文件长度,就可发现该图像是否隐藏秘密信息。

源代码 bmphide.m 如下:

```
clc;
clear;
fid=fopen('baboon.bmp','r');           %读入载体图像文件
[a,length]=fread(fid,inf,'uint8');      %length 是文件的实际长度
fclose(fid);
fid=fopen('baboon.bmp','r');
status=fseek(fid,2,'bof');
fileb=fread(fid,4,'uint8');
filelength=fileb(1)*1+fileb(2)*256+fileb(3)*256*2+fileb(4)*256*3;
%文件图像中保存的文件长度
diff=length-filelength;
%diff 隐藏的信息长度如果相同,表示图像没有隐藏任何信息
fclose(fid);
```

当图像隐藏信息后,如 $\text{diff}=10$,表示隐藏 10 个字节的信息。因此要在图像中隐藏信息,需修改图像文件长度,也就是修改第 3~第 6 字节,如此例中需在图像中隐藏 10 个字节信息,需要将文件长度增加 10。在 Ultraedit 中手工将第 3 个字节由原来的 $0x36$ (十进制的 54),变为 $0x40$ (十进制的 64),然后再运行上述程序,发现 $\text{diff}=0$,表示图像隐藏并修改文件的长度后,通过该方法无法发现图像中是否隐藏信息,同时使用图像查看工具打开图像文件,发现图像在视觉上和原图像没有任何差别。

2. 在文件头与图像数据之间隐藏信息

在数据区开始之前隐藏信息,也就是在 54 和 55 个字节之间隐藏信息,隐藏的秘密信息从 hidden.txt 文件中读取,此种方法修改图像数据的偏移量和图像数据的文件长度。

源代码 bmpheadhiding.m 如下:

```
clc;
clear;
wm=randsrc(1,300,[0 1]);               %产生随机水印
fid=fopen('baboon.bmp','r');           %读入载体图像文件
[a,length]=fread(fid,inf,'uint8');
fclose(fid);
msgfid=fopen('hidden.txt','r');         %打开秘密文件
[msg,count]=fread(msgfid);
fclose(msgfid);
wa=a;
j=1;
wa(11)=54+count;
wa(3)=wa(3)+count;
```

```

for i=55:64
    wa(i)=uint8(msg(j,1));           %隐藏密码信息
    j=j+1;
end
for i=55:length
    wa(i+10)=a(i);
end
figure;
wa=uint8(wa);
fid=fopen('watermarked.bmp','wb');
fwrite(fid,wa);
fclose(fid);
imshow('watermarked.bmp');

```

3. BMP 图像文件隐藏信息的检测

在 BMP 图像中隐藏信息的时候一般都是通过修改文件的偏移量和图像文件中图像的长度来隐藏信息,但在 BMP 图像文件中, $\text{file_length} = \text{biwidth} * \text{biBytecount} * \text{biHeight} + \text{boffbits}$,其中 biwidth , biheight 表示图像文件的宽度和高度, boffbits 表示文件头到实际位图图像数据之间的偏移量。

源代码 `bmphidecheck.m` 如下:

```

clc;
clear;
wm=randsrc(1,300,[0 1]);           %产生随机水印信息
fid=fopen('baboon.bmp','r');        %读入载体图像文件
[a,length]=fread(fid,inf,'uint8');
status=fseek(fid,2,'bof');
fileb=fread(fid,4,'uint8');
filelength=fileb(1)*1+fileb(2)*256+fileb(3)*256^2+fileb(4)*256^3;
%文件图像的理论长度
status=fseek(fid,18,'bof');
b=fread(fid,4,'uint8');
biwidth=b(1)*1+b(2)*256+b(3)*256^2+b(4)*256^3;
status=fseek(fid,22,'bof');
b=fread(fid,4,'uint8');
biHeight=b(1)*1+b(2)*256+b(3)*256^2+b(4)*256^3;
boffbits=54;                        %偏移量
biBytecount=3;                      %24 位真彩色图像为 3
fclose(fid);
diff=length-filelength;

```

通过 `diff` 的不同来比较图像是否在结尾处隐藏了信息,此种方法不能检测对于修改偏

移量的隐藏检测。

4. 在图像文件头和信息头的保留字段中隐藏信息

BMP 图像文件中有很多从不使用的保留字节,如 7、8、9、10 字节是保留的,必须为 0,可在第 7、8、9、10 字节隐藏秘密信息。

源代码 bmpreservedhiding.m 如下:

```
clc;
clear;
fid=fopen('baboon.bmp','r');           %读入载体图像文件
[a,length]=fread(fid,inf,'uint8');
fclose(fid);
wa=a;
%在 BMP 的 7、8、9、10 保留字中隐藏秘密信息 BAPT,ASCII 值为 0x42,0x55,0x50,0x54
wa(7)=66;
wa(8)=85;
wa(9)=80;
wa(10)=84;
figure;
wa=uint8(wa);
fid=fopen('watermarked.bmp','wb');
fwrite(fid,wa);
fclose(fid);
imshow('watermarked.bmp');
```

10.3 LSB 图像信息隐藏

【实验目的】

了解信息隐藏中最常用的 LSB 算法特点,掌握 LSB 算法原理,设计并实现一种基于图像的 LSB 隐藏算法;了解如何通过峰值信噪比来对图像质量进行客观评价,并计算峰值信噪比。

【实验环境】

- (1) Windows XP 或 Vista 操作系统;
- (2) Matlab7.1 科学计算软件;
- (3) BMP 灰度图像文件。

【原理简介】

任何多媒体信息,在数字化时,都会产生物理随机噪声,而人的感观系统对这些随机噪

声不敏感。替换技术就是利用这个原理,通过使用秘密信息比特替换随机噪声,从而完成信息隐藏目标。

BMP 灰度图像的位平面如图 10-7 所示,每个像素值为 8 bit 二进制值,表示该点亮度。

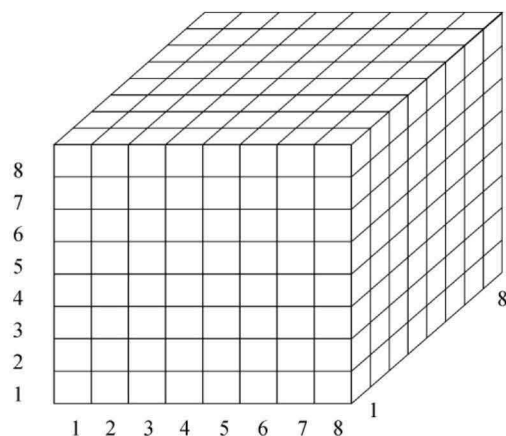


图 10-7 BMP 灰度图像的位平面

不同位平面对视觉影响不同,可用图 10-8~图 10-10 表示。

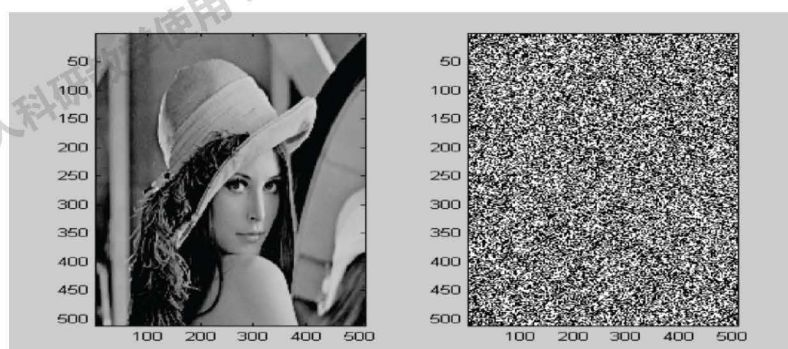


图 10-8 去除第一位平面的 Lena 图像和第一位平面

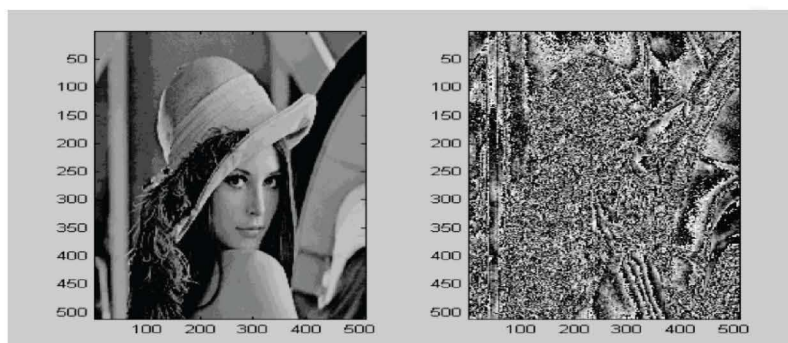


图 10-9 去除第 1~4 位平面的 Lena 图像和第 1~4 位平面

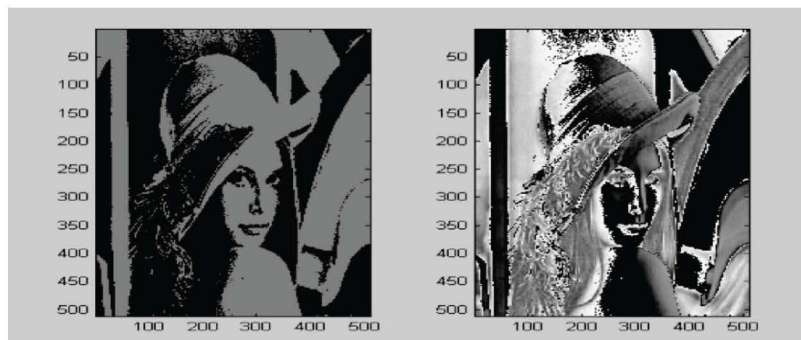


图 10-10 去除第 1~7 位平面的 Lena 图像和第 1~7 位平面

图像高位平面对图像感官质量起主要作用,去除图像最低几个位平面并不会造成画面质量的明显下降。利用这个原理可用秘密信息(或称水印信息)替代载体图像低位平面以实现信息嵌入。

本节中算法选用最低位平面来嵌入秘密信息。最低位平面对图像的视觉效果影响最轻微,但很容易受噪声影响和攻击,解决办法可采用冗余嵌入的方式来增强稳健性。即在一个区域(多个像素)中嵌入相同的信息,提取时根据该区域中的所有像素判断。

1. 隐藏算法

算法分为三个部分实现:

- 隐藏算法;
- 提取算法;
- 测试脚本。

(1) 隐藏算法

源代码 hide_lsb.m 如下:

```
function o = hide_lsb(block, data, I)
%function o = hide_lsb(block, data, I)
%block:隐藏的最小分块大小
%data:秘密信息
%I:原始载体
si = size(I);
lend = length(data);
%将图像划分为M*N个小块
N = floor(si(2) / block(2));
M = min(floor(si(1) / block(1)), ceil(lend / N));
o = I;
for i = 0 : M - 1
%计算每小块垂直方向起止位置
rst = i * block(1) + 1;
red = (i + 1) * block(1);
```

```

for j = 0 : N - 1
%计算每小块隐藏的秘密信息的序号
idx = i * N + j + 1;
if idx > lend
    break;
end;
%取每小块隐藏的秘密信息
bit = data(idx);
%计算每小块水平方向起止位置
cst = j * block(2) + 1;
ced = (j + 1) * block(2);
%将每小块最低位平面替换为秘密信息
o(rst:red, cst:ced) = bitset(o(rst:red, cst:ced), 1, bit);
end;
end;

```

(2) 提取算法

源代码 dh_lsbs.m 如下:

```

function out = dh_lsbs(block, I)
%function out = dh_lsbs(block, I)
%block: 隐藏的最小分块大小
%I: 携密载体
si = size(I);
%将图像划分为 M*N 个小块
N = floor(si(2) / block(2));
M = floor(si(1) / block(1));
out = [];

```

%计算比特 1 判决阈值:即每小块半数以上元素隐藏的是比特 1 时,判决该小块嵌入的信息为 1

```

thr = ceil((block(1) * block(2) + 1) / 2);
idx = 0;
for i = 0 : M - 1
%计算每小块垂直方向起止位置
rst = i * block(1) + 1;
red = (i + 1) * block(1);
for j = 0 : N - 1
%计算每小块图像隐藏的秘密信息序号
idx = i * N + j + 1;
%计算每小块水平方向起止位置
cst = j * block(2) + 1;

```

信息隐藏与数字水印

```
ced = (j + 1) * block(2);
%提取小块最低位平面,统计1比特个数,判决输出秘密信息
tmp = sum(sum(bitget(I(rst:red, cst:ced), 1)));
    if(tmp >= thr)
        out(idx) = 1;
    else
        out(idx) = 0;
    end;
end;
end;
(3) 测试脚本
源代码 test.m 如下:
fid = 1;
len = 10;
%随机生成要隐藏的秘密信息
d = randsrc(1, len, [0 1]);
block = [3, 3];
[fn, pn] = uigetfile({'*.bmp', 'bmp file (*.bmp)';}, '选择载体');
s = imread(strcat(pn, fn));
ss = size(s);
if(length(ss) >= 3)
    I = rgb2gray(s);
else
    I = s;
end;
si = size(I);
sN = floor(si(1) / block(1)) * floor(si(2) / block(2));
tN = length(d);
%如果载体图像尺寸不足以隐藏秘密信息,则在垂直方向上复制填充图像
if sN < tN
    multiple = ceil(tN / sN);
    tmp = [];
    for i = 1:multiple
        tmp = [tmp; I];
    end;
    I = tmp;
end;
%调用隐藏算法,把携秘载体写至硬盘
stegoed = hide_lsblock(block, d, I);
```



```

imwrite(stegoed, 'hide.bmp', 'bmp');
[fn, pn] = uigetfile({'*.bmp', 'bmp file (*.bmp)';}, '选择隐蔽载体');
y = imread(strcat(pn, fn));
sy = size(y);
if(length(sy) >= 3)
    I = rgb2gray(y);
else
    I = y;
end;
%调用提取算法,获得秘密信息
out = dh_lsb(block, I);
%计算误码率
len = min(length(d), length(out));
rate = sum(abs(out(1:len) - d(1:len))) / len;
y = 1 - rate;
fprintf(fid, 'LSB:len:%d\t error rate:%f\t error num:%d\n', len, rate, len*rate);

```

2. 计算峰值信噪比

- 峰值信噪比定义: $PSNR = XY \max_{x,y} \frac{p_{x,y}^2}{\sum_{x,y} (p_{x,y} - \tilde{p}_{x,y})^2}$

- 峰值信噪比函数
- 测试脚本

具体步骤如下。

(1) 峰值信噪比函数

源代码 psnr.m 如下:

```

function y = psnr(org, stg)
% function y = psnr(org, stg)
y = 0;
sorg = size(org);
sstg = size(stg);
if sorg ~= sstg
    fprintf(1, 'org and stg must have same size! \n');
end;
np = sum(sum((org - stg).^2));
y = 10 * log10(max(max(double((org).^2)) * sorg(1) * sorg(2) / np));

```

(2) 测试脚本

```

org = imread('lena.bmp');
stg = imread('hide.bmp');
fprintf(1, 'psnr:%f\n', psnr(org, stg));

```

10.4 DCT 域图像水印

【实验目的】

了解频域水印的特点,掌握基于 DCT 系数关系的图像水印算法原理,设计并实现一种基于 DCT 域的图像水印算法。

【实验环境】

- (1) Windows XP 或 Vista 操作系统;
- (2) Matlab7.1 科学计算软件;
- (3) 图像文件。

【原理简介】

在信号的频域(变换域)中隐藏信息要比在时域中嵌入信息具有更好的鲁棒性。一副图像经过时域到频域的变换后,可将待隐藏信息藏入图像的显著区域,这种方法比 LSB 以及其他一些时域水印算法更具抗攻击能力,而且还保持了对人类感官的不可察觉性。常用的变换域方法有离散余弦变换(DCT)、离散小波变换(DWT)和离散傅里叶变换(DFT)等。

本章介绍一种提取秘密信息的时候不需要原始图像的盲水印算法,算法的思想是利用载体中两个特定 DCT 系数的相对大小来表示隐藏的信息。载体图像分为 8×8 分块,进行二维 DCT 变换,分别选择其中的两个位置,比如用 (u_1, v_1) 和 (u_2, v_2) 代表所选定的两个系数的坐标。如果 $B_i(u_1, v_1) < B_i(u_2, v_2)$, 代表隐藏 1, 如果相反,则交换两系数。如果 $B_i(u_1, v_1) > B_i(u_2, v_2)$, 代表隐藏 0, 如果相反,则交换两系数。提取的时候接收者对包含水印的图像文件进行二维 DCT 变换,比较每一块中约定位置的 DCT 系数值,根据其相对大小,得到隐藏信息的比特串,从而恢复出秘密信息。但是在使用上述算法的过程中,注意到如果有一对系数大小相差非常少,往往难以保证携带图像在保存和传输的过程中以及提取秘密信息的过程中不发生变化。因此在实际的设计过程中,一般都是引入一个 Alpha 变量对系数的差值进行控制,将两个系数的差别放大,可以保证提取秘密信息的正确性。

【实验步骤】

1. 嵌入水印信息

源代码 dcthiding.m 如下:

```
clc;
clear;
msgfid=fopen('hidden.txt','r');           %打开秘密文件,读入秘密信息
[msg,count]=fread(msgfid);
count=count*8;
alpha=0.02;
```

```

fclose(msgfid);
msg=str2bit(msg)';
[ len col]=size(msg);
io=imread('lena.bmp');           %读取载体图像
io=double(io)/255;
output=io;
i1=io(:,:,1);                     %取图像的一层来隐藏
T=dctmtx(8);                       %对图像进行分块
DCTrgb=blkproc(i1,[8 8],'P1*x*P2',T,T'); %对图像分块进行 DCT 变换
[ row,col]=size(DCTrgb);
row=floor(row/8);
col=floor(col/8);
% 顺序信息嵌入
temp=0;
for i=1:count;
    if msg(i,1)==0
        if DCTrgb(i+4,i+1)<DCTrgb(i+3,i+2) %选择(5,2)和(4,3)这一对系数
            temp=DCTrgb(i+4,i+1);
            DCTrgb(i+4,i+1)=DCTrgb(i+3,i+2);
            DCTrgb(i+3,i+2)=temp;
        end
    else
        if DCTrgb(i+4,i+1)>DCTrgb(i+3,i+2)
            temp=DCTrgb(i+4,i+1);
            DCTrgb(i+4,i+1)=DCTrgb(i+3,i+2);
            DCTrgb(i+3,i+2)=temp;
        end
    end
    if DCTrgb(i+4,i+1)<DCTrgb(i+3,i+2)
        DCTrgb(i+4,i+1)=DCTrgb(i+4,i+1)-alpha; %将原本小的系数
调整更小,使得系数差别变大
    else
        DCTrgb(i+3,i+2)=DCTrgb(i+3,i+2)-alpha;
    end
end
%将信息写回并保存

wi=blkproc(DCTrgb,[8 8],'P1*x*P2',T',T); %对 DCTrgb1 进行逆变换
output=io;

```

信息隐藏与数字水印

```
output(:,:,1)=wi;
inwrite(output,'watermarkedlena.bmp');
figure;
subplot(1,2,1);imshow('lena.bmp');title('原始图像');
subplot(1,2,2);imshow('watermarkedlena.bmp');title('嵌入水印图像');
```

2. 提取秘密信息

源代码 dctextract.m 如下:

```
clc;
clear;
wi=imread('watermarkedlena.bmp');           %读取携密图像
wi=double(wi)/255;
wi=wi(:,:,1);                               %取图像的一层来提取
T=dctmtx(8);                                %对图像进行分块
DCTcheck=blkproc(wi,[8 8],'P1*x*P2',T,T'); %对图像分块进行DCT变换

for i=1:80                                   %80为隐藏的秘密信息的比特数
    if DCTcheck(i+4,i+1)<=DCTcheck(i+3,i+2)
        message(i,1)=1;
    else
        message(i,1)=0;
    end
end
out=bit2str(message);
fid=fopen('message.txt','wt');
fwrite(fid,out)
fclose(fid);
```

10.5 回声信息隐藏

【实验目的】

回声隐藏利用人耳听觉系统的时域掩蔽特性,在载体数据的环境特性(回声)中嵌入水印信息。掌握语音的回声隐藏算法原理,设计并实现一种回声隐藏算法。

【实验环境】

- (1) Windows XP 或 Vista 操作系统;
- (2) Matlab7.1 科学计算软件;
- (3) 音频文件。

【原理简介】

音频信号和经过回声隐藏的携密数据对于人耳来说,前者就像是从耳机中听到的声音,没有回声。而后者就像是从扬声器里听到的声音,有所处空间(诸如墙壁、家具等物体)产生的回声。回声隐藏巧妙地利用人类听觉系统(HAS)的时域掩蔽特性,通过向音频信号中引入回声来完成隐藏秘密信息。回声隐藏与其他方法不同,它不是将水印信息当成随机噪声嵌入到载体数据中,而是利用载体数据的环境特征(回声)来嵌入水印信息。尽管引入回声的方法必然会导致载体音频信号的失真,但只要选择合理的回声参数 a 和 m ,附加的回声就难以被人类听觉系统所觉察。回声的数字音频信号可表示为: $y[n] = s[n] + \lambda * s[n-m]$, 其中 $y[n]$ 是加入回声后的音频信号, $s[n]$ 是原始音频信号, λ 为回声的幅度系数, m 为时延参数。 λ 为 $0 \sim 1$ 之间的正数, m 一般表示回声信号滞后于原始信号的样点间隔。由 HAS 的时域后掩蔽特性可知,对于回声时延的大小是有限制的。一般情况下,回声时延 m 的取值在 $50 \sim 200$ ms 之间。过小会增加嵌入信息恢复的难度,过大则会影响隐藏信号的不可感知性。同时,回声的幅度系数 a 的取值也同样需要精心选择,其值与信号传输环境和时延取值有关,一般 λ 取值在 $0.6 \sim 0.9$ 之间。

【实验步骤】

1. 嵌入算法

步骤如下:

(1) 首先将音频采样数据文件分成包含 N 个样点的子帧,子帧的时长可以根据隐藏数据量的大小划分,一般时长从几个毫秒到几十毫秒,每个子帧隐藏一个比特的秘密信息。

(2) 定义两种不同的回声时延 m_0, m_1 (其中, m_0, m_1 均要求远小于子帧时长 N)。当秘密信号比特值为“0”时,回声时延为 m_0 ; 当秘密信号比特值为“1”时,回声时延为 m_1 ;

(3) 将载体信号的每个子帧按照式 $y[n] = s[n] + \lambda * s[n-m]$ 产生回声信号。

(4) 将所有含回声的信号段串联成连续信号。

源代码 echohiding.m 如下:

```
[s,fs,bits]=wavread('1.wav');
[row,col]=size(s)
if(row>col)
    s=s';
end;      %把矩阵转换
msgfid=fopen('hidden.txt','r');
[msg,count]=fread(msgfid);
msg=str2bit(msg);
msg=uint8(msg)';
len=length(s);
i=0;
fragment=8;
N=floor(len/fragment);
```


信息隐藏与数字水印

```
lend = length(msg);
atten=0.9;
d0=100;
d1=200;
s0 = atten * [zeros(1, d0), s(1:len-d0)];    %backward echo with delay 0
s1 = atten * [zeros(1, d1), s(1:len-d1)];    %backward echo with delay 1
o = s0;
for i = 0 : N-1
    if((i + 1) > lend)
        bit = 0;
    else
        bit = msg(i + 1);
    end;
    if bit == 1
        st = i * fragment + 1;
        ed = (i + 1) * fragment;
        o(st : ed) = s1(st : ed);
    end;
end;
o = s + o;
x=0:len-1;
figure;
plot(x,o,x,s);
wavwrite(o,fs,'watermarked.wav');
```

2. 提取算法

回声隐藏算法的最大难点在于秘密信号的提取,其关键在于回声间距的确定。由于回声信号是载体音频信号和引入回声信号的卷积,因此在提取时需要利用语音信号处理中的同态处理技术,利用倒谱相关测定回声间距。在进行提取时,必须要确定数据的起点并预先得到子帧的长度、时延 m_0 和 m_1 等参数值。

(1)将接收到的数据按照预定的时长划分为子帧。

(2)求出各段的倒谱自相关值,比较 m_0 和 m_1 处的自相关幅值 F_0 和 F_1 ,如果 F_0 大于 F_1 ,则嵌入比特值为“0”;如果 F_1 大于 F_0 ,则嵌入比特值为“1”。

源代码 echoextract.m 如下:

```
clc;
clear;
[in,fs,bits]=wavread('watermarked.wav');
[row, col] = size(in);
if(row > col)
    in = in';
end;
```

```

fragment=56;
fft_len = 0;
d0=8;
d1=12;
i = 0;
out = [];
N = floor(length(in) / fragment);
for i = 0 : N-1
    st = i * fragment + 1;
    ed = (i + 1) * fragment;
    p = in(st : ed);
    p = fft(p);
    if p ~ = 0
        p = ifft(log(abs(p)));
    end;
    p = abs(p);
    d11 = p(1 * d1 + 1);
    d01 = p(1 * d0 + 1);
    if d11 > d01
        out(i + 1) = 1;
    else
        out(i + 1) = 0;
    end;
end;
out=out';
msg=out(1:32);
out=bit2str(msg);
fid=fopen('message.txt','wt');
fwrite(fid, out)
fclose(fid);

```

10.6 LSB 信息隐藏的卡方分析

【实验目的】

了解什么是隐写分析(steganalysis),隐写分析与信息隐藏和数字水印的关系。掌握基于图像的 LSB 隐写的分析方法,设计并实现一种基于图像的 LSB 卡方隐写分析方法。

【实验环境】

- (1) Windows XP 或 Vista 操作系统;
- (2) Matlab7.1 科学计算软件;
- (3) 图像文件 man.bmp。

【原理简介】

隐写术和隐写分析技术从本质上来说是互相矛盾的,但是两者实际上又是相互促进的。隐写分析是指对可疑的载体信息进行攻击以达到检测、破坏,甚至提取秘密信息的技术,它的主要目标是为了揭示媒体中隐藏信息的存在性,甚至只是指出媒体中存在秘密信息的可疑性。

图像 LSB 信息隐藏的方法是用嵌入的秘密信息取代载体图像的最低比特位,原来图像的 7 个高位平面与代表秘密信息的最低位平面组成含隐藏信息的新图像。虽然 LSB 隐写在隐藏大量信息的情况下依然保持良好的视觉隐蔽性,但使用有效的统计分析工具可判断一幅载体图像中是否含有秘密信息。

目前对于图像 LSB 信息隐藏主要分析方法有卡方分析、信息量估算法、RS 分析法和 GPC 分析法等。本节介绍卡方分析方法。卡方分析的步骤如下:

设图像中灰度值为 j 的像素数为 h_j ,其中 $0 \leq j \leq 255$ 。如果载体图像未经隐写, h_{2i} 和 h_{2i+1} 的值会相差得很远。秘密信息在嵌入之前往往经过加密,可以看作是 0、1 随机分布的比特流,而且值为 0 与 1 的可能性都是 $1/2$ 。如果秘密信息完全替代载体图像的最低位,那么 h_{2i} 和 h_{2i+1} 的值会比较接近,可以根据这个性质判断图像是否经过隐写。接下来,定量分析载体图像最低位完全嵌入秘密信息的情况。嵌入信息会改变直方图的分布,由差别很大变得近似相等,但是却不会改变 $h_{2i} + h_{2i+1}$ 的值,因为样值要么不改变,要么就在 h_{2i} 和 h_{2i+1} 之间改变。令 $h_{2i}^* = \frac{h_{2i} + h_{2i+1}}{2}$, $q = \frac{h_{2i} - h_{2i+1}}{2}$,显然这个值在隐写前后是不会变的。

如果样值为 $2i$,那么它对参数 q 的贡献为 $1/2$;如果样值为 $2i+1$,那么它对参数 q 的贡献为 $-1/2$ 。载体音频中共有 $2h_{2i}^*$ 个样点的值为 $2i$ 或 $2i+1$,若所有样点都包含 1 bit 的秘密信息,那么每个样点为 $2i$ 或 $2i+1$ 的概率就是 0.5。当 $2h_{2i}^*$ 较大时,根据中心极限定理,式(10-1)成立。

$$\frac{h_{2i} - h_{2i+1}}{\sqrt{2h_{2i}^*}} = \sqrt{2} \cdot \frac{h_{2i} - h_{2i}^*}{\sqrt{h_{2i}^*}} \rightarrow N(0, 1) \quad (10-1)$$

其中, $\rightarrow N(0, 1)$ 表示近似服从正态分布。因此

$$r = \sum_{i=1}^k \frac{(h_{2i} - h_{2i}^*)^2}{h_{2i}^*} \quad (10-2)$$

服从卡方分布。式(10-2)中, k 等于 h_{2i} 和 h_{2i+1} 所组成数字对的数量, h_{2i}^* 为 0 的情况不计在内。 r 越小表示载体含有秘密信息的可能性越大。结合卡方分布的密度计算函数计算载体被隐写的可能性为

$$p = 1 - \frac{1}{2^{\frac{k-1}{2}} \Gamma\left(\frac{k-1}{2}\right)} \int_0^r \exp\left(-\frac{t}{2}\right) t^{\frac{k-1}{2}-1} dt \quad (10-3)$$

如果 p 接近于 1, 则说明载体图像中含有秘密信息。

【实验步骤】

1. LSB 嵌入和直方图变化

对图像进行 LSB 嵌入, 比较嵌入秘密信息前后的直方图变化。

源代码 hist_change.m 如下:

```
[fn, pn] = uigetfile({'*.jpg', 'JPEG files (*.jpg)'; '*.bmp', 'BMP files (*.bmp)'});
name = strcat(pn, fn);
I = rgb2gray(imread(name)); %对灰度图像进行隐藏
sz = size(I);
% generate msg
rt = 1; %隐写率为 100%
row = round(sz(1) * rt);
col = round(sz(2) * rt);
msg = randsrc(row, col, [0 1; 0.5 0.5]);
% LSB hide
stg = I;
stg(1:row, 1:col) = bitset(stg(1:row, 1:col), 1, msg);
nI = sum(hist(I, [0:255]), 2)';
nS = sum(hist(stg, [0:255]), 2)';
x = [80 : 99];
figure;
stem(x, nI(81:100)); figure;
stem(x, nS(81:100));
```

2. 卡方分析函数

源代码 StgPrb.m 如下:

```
function p = StgPrb(x)
% 对数据进行分析, 对一个二维数组进行分析, 数组里面的值在 0~255 之间
n = sum(hist(x, [0:255]), 2);
h2i = n([3:2:255]);
h2is = (h2i + n([4:2:256])) / 2;
filter = (h2is ~= 0);
k = sum(filter);
idx = zeros(1, k);
for i = 1 : 127
    if filter(i) == 1
        idx(sum(filter(1:i))) = i;
    end;
```

```

end;
% compute statistics
r = sum(((h2i(idx) - h2is(idx)) .^ 2) ./ (h2is(idx)));
% compute probability
p = 1 - chi2cdf(r, k - 1);
% p = chi2cdf(r, k);
3. LSB 卡方分析源代码
源代码 test.m 如下:
clear all;
[fn, pn] = uigetfile({'*.jpg', 'JPEG files (*.jpg)'; '*.bmp', 'BMP files (*.bmp)'}), 'Select file to hide');
name = strcat(pn, fn);
t = imread(name);
I = t(1:512, 1:512);
sz = size(I);
for k = 1 : 3
    % 根据隐写率大小生成秘密信息, 隐写率为 0.3, 0.5, 0.7 三种
    rt = 0.3 + 0.2 * (k - 1);
    row = round(sz(1) * rt);
    col = round(sz(2) * rt);
    msg = randsrc(row, col, [0 1; 0.5 0.5]);
    % LSB 隐写
    stg = I;
    stg(1:row, 1:col) = bitset(stg(1:row, 1:col), 1, msg);
    imwrite(stg, strcat(pn, sprintf('stg_%d_', floor(100 * rt))),
fn), 'bmp');
    % loop, select certain percent range of stegoed picture to analysis
    i = 1;
    for rto = 0.1 : 0.01 : 1
        row = round(sz(1) * rto);
        col = round(sz(2) * rto);
        % p(ceil(10 * rto)) = StgPrb(stg(1:row, 1:col));
        p(k, i) = StgPrb(stg(1:row, 1:col));
        i = i + 1;
    end;
end;
end;

```


10.7 简单扩频语音水印算法

【实验目的】

了解扩频通信原理,掌握扩频水印算法的基本原理,设计并实现一种基于音频的扩频水印算法,了解参数对扩频水印算法性能的影响。

【实验环境】

- (1) Windows XP 或 Vista 操作系统;
- (2) Matlab7.1 科学计算软件;
- (3) WAV 音频文件。

【原理简介】

扩频是一种能在高噪声环境下可靠传输数据的重要通信技术,其基本原理是:信号在大于所需的带宽内进行传输,数据的带宽扩展是通过一个与数据独立的码字完成的,并且在接收端需要该码字的一个同步接收,以进行解扩和数据恢复。扩频通信的特点是:占据频带很宽,每个频段上的能量很低;即使几个频段的信号丢失,仍可恢复信号;利用相互正交的扩频码,可以在一个宽频带内同时传输很多路信号。扩频通信具有拦截概率小,抗干扰能力强的优点。可以利用这个优点设计水印算法。本例中设计一种简单的算法:利用正交的 PN 序列代表 0、1 信号,并将其叠加到信号 DCT 域。提取水印时,利用 PN 序列的正交性可以较为准确的恢复水印。

【实验步骤】

算法分为四个部分实现:

- PN 产生函数;
- 嵌入算法;
- 提取算法;
- 测试脚本。

具体实现方法如下。

1. PN 产生函数

源代码 pn_gen.m 如下:

```
function out = pn_gen(g, init, shift)
%function out = pn_gen(g, init, shift)
%g:generating ploynomial(msb, lsb)
%init:initial state
%shift:output selector
format = 1;
```

```

out_len = 0;
in_len = 0;
out = [];
%check parameter format, ether g2 = [1 0 0 0 0 0 1 0 1] or g1 = [8 2 0]
tp = max(g);
if tp == 1
    format = 2;      %format of parameter
    in_len = length(g) - 1;
else
    format = 1;
    in_len = g(1);
end;
out_len = 2 ^ in_len - 1; %length of output
out = zeros(1, out_len);
for n = 1 : out_len
    out(n) = init(in_len);
    if format == 1
        tp = 0;
        for m = 2 : length(g)
            tp = mod((tp + init(g(m) + 1)), 2);    %caculate new init(1)
            tp = mod((tp + init(in_len - g(m))), 2); %caculate new init(1)
        end;
    else
        tp = init .* g(2 : (in_len + 1));
        tp = mod(sum(tp), 2);
    end;
    init = [tp init(1 : (in_len - 1))];
end;
for n = (shift - 1) : -1 : 0
    out = [out(2 : out_len), out(1)];
end;

```

2. 隐藏算法

源代码 hide_ds.m 如下:

```

function o = hide_ds(fragment, data, s, atten, pn0, pn1)
[ row, col ] = size(s);
if (row > col)
    s = s';
end;
i = 1;

```

```

n = min(floor(length(s) / fragment), length(data));
o = s;
len = length(pn0);
base = fragment - len + 1;
for i = 1 : n
    st = (i - 1) * fragment + 1;
    ed = i * fragment;
    tmp = dct(s(st:ed));
    atten1 = atten * max(abs(tmp));
    if data(i) == 1
        tmp(base:fragment) = tmp(base:fragment) + atten1 * pn1;
    else
        tmp(base:fragment) = tmp(base:fragment) + atten1 * pn0;
    end;
    o(st : ed) = idct(tmp);
end;

```

3. 提取算法

源代码 dh_ds.m 如下:

```

function out = dh_ds(fragment, in, pn0, pn1)
%out = dh_cepstr_one_pn(d0, d1, fragment, in, exp, win_type, fft_len, data, s, pn)
%decode using cepstrum
%d0:delay 0
%d1:delay 1
%fragment:fragment
%in:blendation
%exp:weight exponent
%win_type:type of window
%fft_len:length of fft
%data:original bit
%s:original voice
%pn : pn sequences
[row, col] = size(in);
if(row > col)
    in = in';
end;
i = 1;
len = floor(length(in) / fragment);
out = [];
len_pn = length(pn0); %length of pn

```

信息隐藏与数字水印

```
base = fragment - len_pn + 1;
for i = 1 : len
    st = (i - 1) * fragment + 1;
    ed = i * fragment;
    p = dct(in(st : ed));
    t0 = sum(p(base:fragment) .* pn0);
    t1 = sum(p(base:fragment) .* pn1);
    if t1 > t0
        out(i) = 1;
    else
        out(i) = 0;
    end;
end;
```

4. 测试脚本

测试步骤如下：

- (1) 选择载体音频；
- (2) 产生水印或秘密信息(例如,每 256 个样点嵌入 1 bit 信息,由载体大小计算最多可嵌入多少比特秘密信息)；
- (3) 产生 PN 序列；
- (4) 选择嵌入强度,嵌入水印；
- (5) 保存携带水印的音频,可利用音频处理软件对音频进行格式转换、重采样等攻击,观察攻击后水印的恢复情况；
- (6) 选择携带水印的音频；
- (7) 提取水印；
- (8) 计算误码率。

源代码 test.m 如下：

```
% 1 select cover audio
[fname, pname] = uigetfile('*.wav', 'Select cover audio');
sourcename = strcat(pname, fname);
s = wavread(sourcename)';
s_len = length(s);
% 2 generate msg to be embedded
frag = 256;
msg_len = floor(s_len / frag);
msg = randsrc(1, msg_len, [0 1]);
% 3 generate PN
degree = 7;
pn0 = 2 * pn_gen([degree 6 0], [zeros(1, degree - 1) 1], 0) - 1;
pn1 = 2 * pn_gen([degree 6 0], [zeros(1, degree - 1) 1], 1) - 1;
```

```
% 4 embed msg
atten = 0.005;
bld = hide_ds(frag, msg, s, atten, pn0, pn1);
% 5 save the stegoed-audio
wavwrite(bld, 8e+3, 'hide.wav');
% 6 select stegoed-audio
[fname, pname] = uigetfile('*.wav', 'Select stegoed-audio');
sourcename = strcat(pname, fname);
steg = wavread(sourcename)';
% 7 extract msg
out = dh_ds(frag, steg, pn0, pn1);
% 8 compute ebr
fid = 1;
ebr = sum(abs(msg - out)) / s_len;
fprintf(fid, 'ebr:%f\n', ebr);
```

仅供个人科研教学使用！